

Neural Network Computation: Model Equations and Runtime Behavior

Suppose you launch a training job.

Everything looks normal...

- ▶ The code runs.
- ▶ The loss decreases.
- ▶ The GPU is being used.
- ▶ The model produces predictions.

But then:

But is it working correctly and efficiently?

Symptoms of Badly Performing Training Runs

Run A

- ▶ Training starts normally
- ▶ GPU memory increases
- ▶ CUDA out of memory
- ▶ Failure occurs during backward

Run B

- ▶ GPU utilization: 25%
- ▶ GPU memory: 60%
- ▶ Training is much slower than expected

Run C

- ▶ Loss decreases normally
- ▶ Suddenly becomes NaN
- ▶ Model never recovers

Run D

- ▶ Loss barely changes
- ▶ Training accuracy remains near chance
- ▶ GPU utilization: 90%

Symptoms of Badly Performing Training Runs

Run A: Memory failure

- ▶ Training starts normally
- ▶ GPU memory increases
- ▶ CUDA out of memory
- ▶ Failure occurs during backward

Run B: Performance Problem

- ▶ GPU utilization: 25%
- ▶ GPU memory: 60%
- ▶ Training is much slower than expected

Run C: Numerical stability problem

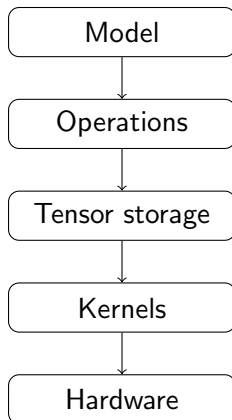
- ▶ Loss decreases normally
- ▶ Suddenly becomes NaN
- ▶ Model never recovers

Run D: Learning/correctness problem

- ▶ Loss barely changes
- ▶ Training accuracy remains near chance
- ▶ GPU utilization: 90%

The Systems View of Training

A training run crosses several layers:



Debugging principle

A runtime symptom is often several layers removed from its root cause.

Using the Stack to Debug

A symptom tells us what we observed, not why it happened.

Debugging question	Where do we look?
What is the model computing?	Model → Operations
How much computation is required?	Operations
What must be stored?	Operations → Tensors
Where is the time going?	Tensors → Kernels → Hardware
What numerical representation is used?	Tensors → Kernels → Hardware

Debugging mindset

Symptom → Question → Layer → Hypothesis

Tensor Vs Kernel

Tensor: A multidimensional array of values

Examples $x \in \mathcal{R}^n$, $W \in \mathcal{R}^{m \times n}$, $X \in \mathcal{R}^{B \times n}$

A tensor has a shape, a data type, storage location, memory cost

Kernel: A compiled function executed on an accelerator to perform an operation on tensors.

Examples: matrix-multiplication kernel, activation kernel, reduction kernel

Neuron Computation

The familiar neural computation:

$$y = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where

- ▶ $\mathbf{x}$ = input
- ▶ $\mathbf{W}$ = weights
- ▶ $\mathbf{b}$ = bias
- ▶ f = activation function
- ▶ y = output

Systems question

What does this equation imply about computation and memory?

Dimensions of a NN

Suppose

$$\mathbf{W} \in \mathbb{R}^{m \times n}, \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{b} \in \mathbb{R}^m$$

Then

$$\mathbf{W}\mathbf{x} \in \mathbb{R}^m$$

and

$$y = f(\mathbf{W}\mathbf{x} + \mathbf{b}) \in \mathbb{R}^m$$

For a batch of B examples:

$$Y = f(\mathbf{W}X + \mathbf{b})$$

with

$$X \in \mathbb{R}^{n \times B}, \quad Y \in \mathbb{R}^{m \times B}.$$

One Neuron: How Much Work?

For one neuron:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

The weighted sum requires approximately:

n multiplications + $(n - 1)$ additions

plus the activation.

For m output neurons:

$$\text{MACs} \approx mn$$

where MAC = multiply-accumulate.

A common approximation:

$$\text{FLOPs} \approx 2mn$$

The Matrix Multiply: Most compute heavy operation

For

$$Y = WX$$

with

$$W \in \mathbb{R}^{m \times n}, \quad X \in \mathbb{R}^{n \times B},$$

we have:

$$Y \in \mathbb{R}^{m \times B}$$

and approximately

$$Bmn \text{ MACs}$$

or

$$2Bmn \text{ FLOPs}$$

Example: A Single Linear Layer

Consider:

`Linear(4096, 4096)`

with batch size

$$B = 32.$$

Then approximately:

$$Bmn = 32 \times 4096 \times 4096$$

MACs.

≈ 537 million MACs

or approximately

1.07 billion FLOPs

Question

Q:

How many such layers can a modern GPU execute per second?

FLOPs Are Not the Whole Story

Two operations can perform similar amounts of arithmetic and have very different runtimes.

Why?

- ▶ How much data must be read?
- ▶ How much data must be written?
- ▶ Can computation be parallelized?
- ▶ How large are the kernels?
- ▶ How often does data move between memory levels?

First systems lesson

Work is not the same thing as **time**.

Parameters Are Data

For

$$W \in \mathbb{R}^{m \times n},$$

the number of weight parameters is

$$mn.$$

Including bias:

$$N_{\text{params}} = mn + m$$

If each parameter uses s bytes:

$$M_{\text{weights}} = (mn + m)s$$

For FP32:

$$s = 4 \text{ bytes.}$$

For FP16/BF16:

Example

For

$$W \in \mathbb{R}^{4096 \times 4096}$$

we have:

$$4096^2 = 16,777,216$$

parameters.

Memory for weights alone:

Format	Weight memory
FP32	≈ 67 MB
FP16/BF16	≈ 34 MB

But training needs more than weights.

One Training Iteration/Loop

Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update

Forward: $x \rightarrow \hat{y}$

Loss: $L(\hat{y}, y)$

Backward: $\frac{\partial L}{\partial W}$

Update: $W \leftarrow W - \text{update}$

Why Did I Get OOM?

CUDA out of memory

Suppose:

GPU capacity = 24 GB

and

model weights = 8 GB.

A common first thought:

“I have 16 GB left.”

But that is not the training memory budget.

Training requires additional storage.

Training Memory Budget

A useful first-order model:

$$M_{\text{training}} \approx M_{\text{parameters}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}} + M_{\text{temporary}}$$

- ▶ Parameters: model weights
- ▶ Gradients: needed for updates
- ▶ Optimizer state: e.g., Adam moments
- ▶ Activations: intermediate values needed for backward
- ▶ Temporary/workspace: kernels and runtime

Debugging principle

OOM means **some component of the memory budget exceeded capacity.**

Backward needs some Forward Values

$$x \rightarrow z = Wx + b \rightarrow a = f(z) \rightarrow L$$

$$\frac{\partial L}{\partial a} \leftarrow \frac{\partial L}{\partial z} \leftarrow \frac{\partial L}{\partial W} \partial W$$

Some forward values must remain alive during backward.

Optimizer State: Adam as an Example

gradient $\rightarrow$ optimizer $\rightarrow$ parameter update

GD: $W \leftarrow W - \eta \delta_W L$ Suppose parameters, gradients, and optimizer states are all stored in FP32.

For each parameter:

$$\underbrace{4}_{\text{parameter}} + \underbrace{4}_{\text{gradient}} + \underbrace{4}_m + \underbrace{4}_v = 16 \text{ bytes}$$

So a rough estimate is:

$$M_{\text{param-related}} \approx 16N$$

for N parameters.

For

$$N = 1 \text{ billion,}$$

this is approximately

16 GB

The Hidden Variable: Activations

Activation memory depends on much more than parameter count.

It depends on:

- ▶ Batch size
- ▶ Sequence length
- ▶ Hidden dimension
- ▶ Number of layers
- ▶ Data type
- ▶ Which intermediates are saved

For a rough activation tensor:

$$X \in \mathbb{R}^{B \times S \times H}$$

the number of elements is

$$BSH$$

and memory is

$$BSH \times \text{bytes per element}$$

Why Sequence Length Can Hurt

Suppose:

$$B = 16, \quad H = 4096$$

and compare:

$$S = 2048 \quad \text{vs.} \quad S = 4096.$$

The activation tensor size changes from:

$$16 \times 2048 \times 4096$$

to

$$16 \times 4096 \times 4096.$$

Therefore:

doubling $S \Rightarrow$ doubling this activation storage

But some operations are worse

OOM Is a Debugging Question

Instead of asking:

“How do I get rid of the OOM?”

ask:

What consumed the memory budget?

Symptom	First hypothesis
OOM immediately	Model / parameters
OOM after forward	Activations
OOM during backward	Saved activations / backward workspace
OOM at optimizer step	Optimizer state / temporary memory
OOM after many iterations	Possible tensor/graph retention

An OOM Investigation

Suppose:

- ▶ GPU memory = 24 GB
- ▶ Model weights = 7 GB
- ▶ Forward succeeds
- ▶ Backward causes OOM

What would you investigate?

1. Activation memory
2. Backward workspace
3. Batch size
4. Sequence length
5. Tensor dtype
6. Retained computation graphs

Engineering rule

Do not optimize blindly. Identify the memory consumer first.

What Should We Measure?

For a real training run, useful observations include:

- ▶ Peak GPU memory
- ▶ Allocated vs. reserved memory
- ▶ Model parameter count
- ▶ Activation sizes
- ▶ Batch size
- ▶ Sequence length
- ▶ Tensor dtypes
- ▶ Optimizer state

The key habit

Turn a vague symptom into a measurable resource question.

Three Ideas to Take Away

1. A neural-network equation represents a **concrete computational workload**.
2. FLOPs tell us about arithmetic work, but **FLOPs alone do not determine runtime**.
3. Training memory is much more than model parameters:

parameters + gradients + optimizer + activations + workspace

Next: How do we find where the computation is actually going?